

PROCESS PLAYBOOK

Pixel Setup in Adobe Tags

A step-by-step guide for the media pixel team

Who this is for	Media and campaign team members who set up and maintain tracking pixels. No coding background is needed.
What you will do	Create and test pixel rules in Adobe Tags (earlier known as Adobe Launch).
Document owner	[Add name / team]
Version and date	1.0 October 2026

1. About this playbook

What is a third-party pixel?

A pixel is a small piece of tracking that tells a marketing or measurement vendor when a customer does something on our website. It can measure a page visit, interaction or conversion. The media team uses this information to measure campaign performance. Different vendors may use a JavaScript library, a tracking request or an approved extension.

Why is this playbook needed?

This playbook explains how to create and test pixel rules in Adobe Tags. It starts with the common steps, then uses BridgeTrack as the first documented vendor. The same process can support other vendors by checking their own events, values and implementation method.

How to use this playbook

- Read sections 1 to 3 once, so you understand the basics.
- For BridgeTrack, follow Parts A to E in order, one step at a time. Use the common guidance for future vendor requirements.
- Screenshots are samples. Your screen may look slightly different, but the field names will match.
- If a step says to use the approved code, ask your technical lead for the latest copy. Do not type code from memory.

NOTE | Naming

Adobe Tags is the new name for Adobe Launch. You may see both names in older documents. They mean the same tool.

Words you will see

Word	What it means in simple terms
Adobe Tags	The tool where we create and manage website tags and pixels. Adobe Launch is the earlier name.
Property	A container in Adobe Tags that holds the rules, Data Elements and extensions for a website.
Rule	An instruction that says when something happens, and the required conditions are true, perform an action.
Event	The trigger that starts a rule, such as a page finishing loading.
Condition	A check that must pass before the action runs, such as the page address or a business value.
Action	What the rule does, such as running a vendor extension action or Custom Code.
Direct Call	A named signal sent by the website that can trigger a matching rule.

Word	What it means in simple terms
Data Element	A saved lookup of a value, such as a campaign code from the web address.
Custom Code	An action or lookup containing JavaScript. Use the approved code from the technical lead.
Library	A package of rule, Data Element and extension revisions built for an environment.
Data Layer	Page or application information supplied by the website, such as window._dl.
Conversion Event	A business action to measure, such as an application submission or approval.
Vendor Pixel	The vendor tag or request used to measure the required event.
Ensign	The earlier tag management tool used for the migrated implementation.

2. How the pixel works

Every rule has three parts

Think of a rule as a simple sentence. Each Adobe Tags rule is built from the same three parts.

WHEN (Event)	IF (Conditions)	THEN (Action)
The page finishes loading, or the website sends a named Direct Call.	The required page, campaign or application values match.	Run the vendor extension action or Custom Code.

Two ways a pixel gets triggered

Requirement	What starts it	Example
Page-load pixel	An appropriate page-load event, such as Window Loaded.	Run the tag when the required page has loaded.
Application or business event	The website sends the exact named Direct Call.	Run the matching tag when the application reports that event.

From event to action

The event decides when the rule is checked. The conditions decide whether its action should run.

1. The page loads or the application sends the required event.
2. The matching Launch rule checks its conditions.
3. The configured action runs when the conditions qualify.
4. The vendor request is created when any checks inside the action also qualify.

3. Before you start

Check every item below before you open Adobe Tags. This saves time and avoids rework.

Done	What you need	Why
[]	Access to the correct Adobe Tags property with rule-editing permission	You need the correct property and permissions.
[]	Access to a Development library	New work is built and tested in Development first.
[]	Approved requirement: vendor, event, conditions and parameters	It tells you what to configure.
[]	Approved vendor extension or code, where required	Use the method required by that vendor.
[]	A UAT page or journey and a Chrome or Edge browser	Used for testing in Part E.
[]	Names and settings of the Data Elements to reuse	Reuse avoids duplicate lookups (Part D).

DO NOT

Do not change the vendor URL, parameter names or conversion values unless the business requirement calls for it.

Do not copy a rule from another vendor and assume it fits. Check each value first.

4. Vendor 1: BridgeTrack

The BridgeTrack migration retains the vendor request and conversion values while replacing Ensighten-specific methods with Adobe Launch and native JavaScript. Parts A to E explain the current implementation.

What changed from Ensighten

Task	Ensighten	Adobe Tags
App Start page event	Binding on DOM loaded	Core Window Loaded. DOM Loaded and Window Loaded are different events.
Named business event	<code>Bootstrapper.ensEvent.trigger("name")</code>	<code>_satellite.track("name")</code> and a matching Direct Call event
Insert the vendor script	<code>Bootstrapper.insertScript(url)</code>	Native script creation and appendChild (Part A)
Application page-view signal	<code>na_tms_page_view</code>	<code>na_launch_page_view</code>
Application interaction signal	<code>na_tms_custom_event</code>	<code>na_launch_interaction</code>
Conditions and checks	Ensighten binding and action checks	Launch rule conditions and checks inside Custom Code

IMPORTANT | What stays the same

The BridgeTrack web address, parameter names and conversion values (docid) are not changed. We only replace the Ensighten-specific parts.

The BridgeTrack request in plain words

Every BridgeTrack pixel is a web request sent to this address:

```
https://citi.bridgetrack.com/track/citicards/s/
```

The request carries a few values. Each value has a fixed name that BridgeTrack expects. You keep these names exactly as they are.

Parameter	Source or meaning
pid	Existing prodId + prodType sequence in the URL expression. Preserve its order and value types.
docid	BridgeTrack conversion identifier; App Start uses NGA_AppPage.
app	appType
appld	Application ID from appld
type	businessTypCd
r	Math.random() * 10000000000000; a random cache-busting value
ProspectID	prspectId; preserve this exact parameter spelling and casing
sc	Rule-specific sc value; App Start reads window.sc

Retain the URL, parameter casing and concatenation. The r value is a cache buster, not a guaranteed unique conversion or business identifier.

5. Part A: Set up the App Start pixel

The App Start pixel tells BridgeTrack that a customer landed on the application page. Its docid is **NGA_AppPage**. It is the most detailed pixel, so we use it as the main example. Once you understand it, the other pixels are easier.

NOTE | In one line

Event: Window Loaded. Conditions: right page, right application type, right campaign. Action: Custom Code that waits for six values, then sends the pixel.

Step A1 Create the rule

1. Sign in to Adobe Tags and open the correct property.
2. Select Rules in the left menu, then select Create New Rule.
3. Create or open the App Start rule. The existing rule is named Third_Party_Bridgetrack_MV_Cards_Conversion_NGAAppStartPages.

Step A2 Add the event: Window Loaded

This makes the rule start when the page has fully loaded.

1. Under Events, select Add.
2. Set Extension to **Core**.
3. Set Event Type to Window Loaded and event order to 50.
4. Select Keep Changes.

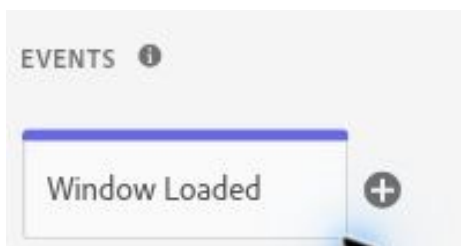


Figure 1. App Start Core Window Loaded event.

Step A3 Add the conditions

The rule runs only when all conditions below are true. Add each one under Conditions, using the Core extension.

Condition	Launch configuration
Hostname	Core Custom Code using the existing approved UAT/online hostname condition. Retain that condition rather than inventing a new host list.
Application path	Core Path condition with regular expression enabled: ^(/US/(ag nga)/cards/application/?)\$
Application query	Core Query String condition: parameter app; regular expression ^(UNSOL)\$
Campaign value	Core Value Comparison: %URL_cmp% Starts With afa. This is a separate value check, using the lowercase prefix.

The path condition covers /US/ag/cards/application and /US/nga/cards/application, with an optional trailing slash. The expression is a pattern; it is not a literal application URL.

URL_cmp reads the query parameter cmp. Its parameter-name case setting is separate from the rule's comparison with afa. See Part D for the lookup configuration.

TIP

Retain the existing approved hostname condition. Use the conditions required by each rule; do not apply App Start conditions to every BridgeTrack tag.

Step A4 Add the action: Custom Code

1. Under Actions, select Add.
2. Set Extension to **Core**.
3. Set Action Type to **Custom Code**.
4. Select Open Editor.
5. Paste the approved App Start code from your technical lead.
6. Select Save, then Keep Changes.

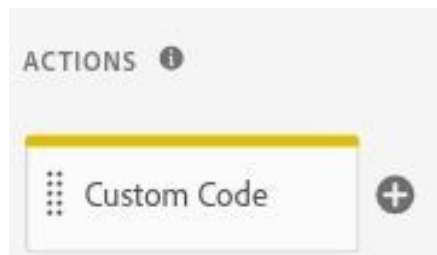


Figure 2. App Start Core Custom Code action.

NOTE

Conditions in the Launch interface and checks inside Custom Code are separate. Both must be understood.

Step A5 Understand what the code does

You do not need to write this code. This section only helps you recognise the main parts, so you can check that the right code is pasted.

Part 1: It collects the values

The local `_dl` object is read inside `checker()`, so the application values are read again on each check. A missing `window._dl` is handled as an empty object in this action.

```
var _dl = window._dl || {};
var prodId = _dl.pid || '';
var prspectId = _dl.prospect_id || '';
```

```
1 var retries = 0;
2 var maxRetries = 10; // to avoid infinite loop, max 10 seconds
3
4 function checker() {
5   var _dl=window._dl||{};
6   var prodId=_dl.pid||'';
7   var prspectId=_dl.prospect_id||'';
8   var sc = window.sc ||'';
9   var appId=window.appId||'';
10  var businessTypCd=window.businessTypCd||'';
11  var prodType=window.prodType||'';
12  var appType=window.appType||'';
13
14  var allReady = prodId && prspectId && sc && businessTypCd && prodType && appType;
15  if (allReady) {
```

Figure 3. App Start input values and six-field readiness check.

Part 2: It waits until all six values are ready

App Start requires six truthy values: `prodId`, `prspectId`, `sc`, `businessTypCd`, `prodType` and `appType`. The source table also shows `appId`, which is used in the request.

Value used by App Start	Actual source
prodId	window._dl.pid
prspectId	window._dl.prospect_id
sc	window.sc
businessTypCd	window.businessTypCd
prodType	window.prodType
appType	window.appType
appld	window.appld; included in the URL, outside the six-field readiness check

The first check runs immediately. If any required value is missing, another check is scheduled after approximately one second. The action supports up to ten scheduled retries after the first check, giving up to eleven evaluations. Once all six values are present, it constructs the URL and inserts the script. If a value remains missing, it stops after the configured limit and logs a warning. Browser timing can affect the elapsed time.

```
var allReady = prodId && prspectId && sc &&
  businessTypCd && prodType && appType;
```

NOTE | appld

appld is included in the BridgeTrack URL but is not part of allReady. The current action can send an empty appld when the six required values are available. Preserve this existing behaviour.

Part 3: It sends the pixel

Once all values are ready, the code builds the BridgeTrack web address and sends it. This is the part that replaced the Ensign helper. See the next step.

Step A6 Check the script insertion lines

Bootstrapper.insertScript() belongs to Ensign. It must not be called by the migrated Launch action, because it depends on the old tool.

DO NOT | Old line, remove it

```
Bootstrapper.insertScript(url);
```

In Adobe Tags, the code uses five plain browser lines instead. Check they are present in the pasted code.

```
var script = document.createElement("script");
script.type = "text/javascript";
script.async = true;
script.src = url;
(document.head || document.documentElement).appendChild(script);
```

```
18  var script = document.createElement("script");
19  script.type = "text/javascript";
20  script.async = true;
21  script.src = url;
22  (document.head || document.documentElement).appendChild(script);
23  console.log("Pixel fired via recursive checker:", url);
```

Figure 4. Native JavaScript script insertion in the App Start action.

Line	What it does
document.createElement("script")	Creates a script element.
script.type	Sets the JavaScript type.
script.async = true	Loads the script asynchronously.
script.src = url	Sets the BridgeTrack vendor URL.
appendChild(script)	Appends the element to head or documentElement and starts the request.

Bootstrapper.insertScript() belongs to Ensign. Launch uses native JavaScript to create the script, set its type and async flag, assign the URL and append it to the head or document element. This removes the Ensign dependency.

The Ensign helper deferred insertion before the first head child; Launch appends immediately. The vendor URL and business parameters remain unchanged.

NOTE | The last line in the screenshot

The line starting with `console.log` writes the message "Pixel fired via recursive checker" in the browser. It is useful during testing (see Part E).

Step A7 Save the rule

1. Select Save to Library (or Save, if your team uses another flow).
2. Add the rule to your Development library.
3. Build Development with the rule and its dependencies included. Then move to Part E to test.

IMPORTANT

Do not publish to Production until testing in Part E is complete and approved by your team.

6. Part B: Set up the outcome pixels

The outcome pixels tell BridgeTrack the result of an application: Approved, Declined, Pending or Registered. Prequalification has separate Submit, Approved and Declined events. Each event has its own rule and docid. These actions construct the request directly rather than using App Start's retry function.

Which pixel uses which signal

Business event	Direct Call identifier	BridgeTrack docid
Approved	NGA_application_approved	NGA_Approved
Declined	NGA_application_declined	NGA_Declined
Pending	NGA_application_pended	NGA_Pending
Registered	NGA_application_registration	NGA_Registered
Prequal Submit	NGA_prequal_submit	NGA_INPQ_SUBMIT
Prequal Approved	NGA_prequal_approved	NGA_INPQ_APPROVE
Prequal Declined	NGA_prequal_declined	NGA_INPQ_DECLINE

NOTE | Outcome and prequalification events

Each application or prequalification event uses its own Direct Call and docid. Use the identifier exactly as shown, including capitalization and the existing "pended" and "registration" wording. App Start's readiness and retry function is not automatically required by these rules.

Step B1 Create one rule per outcome

1. Select Rules, then Create New Rule.
2. Name the rule clearly. Example: **BridgeTrack - Approved**.

Step B2 Add the event: Direct Call

1. Under Events, select Add.
2. Set Extension to **Core**.
3. Set Event Type to **Direct Call**.
4. In Identifier, enter the exact name from the table above. Use event order 50. Copy and paste the identifier.
5. Select Keep Changes.

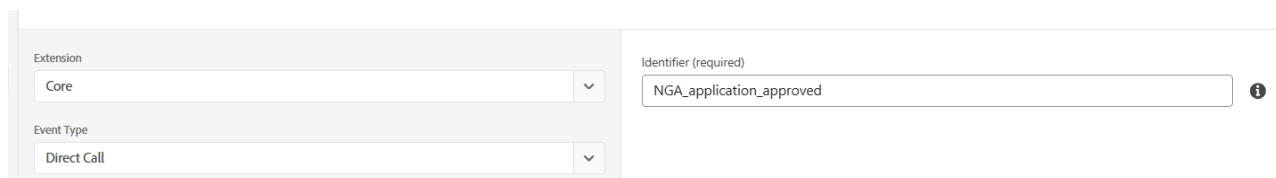


Figure 5. Core Direct Call configured with NGA_application_approved.

IMPORTANT | The identifier must match exactly

A rule name containing Approval does not trigger the tag. The matching Direct Call identifier must be sent by the application. The applicable rule conditions and code checks must also qualify.

Watch for extra spaces, spelling and capital letters.

Step B3 Add conditions where the requirement asks for them

Use the conditions needed by that rule. The existing seven outcome/prequalification rules do not contain outer Launch conditions. Some actions contain their own business checks: the Approval action checks `_dl.site_events` for `app_approved`, `registration_complete` or `application_complete` before building its request.

Step B4 Add the action: Custom Code

1. Under Actions, select Add. Set Extension to **Core** and Action Type to **Custom Code**.
2. Paste the approved code for this pixel from your technical lead.
3. Check that the code has the right docid, for example **NGA_Approved**.
4. Check that the five script insertion lines from Part A, Step A6 are used, and that **Bootstrapper.insertScript** is not present.
5. Select Save, then Keep Changes.

Add the approved Custom Code for that event. These rules read `_dl.pid` and `_dl.prospect_id` and the required global application values, then build the URL and insert the script directly. They do not include App Start's recursive checker.

Step B5 Save and move to testing

Save the rule to the Development library, build it, and continue to Part E. Repeat Part B for each outcome.

Funding Start uses a separate page-load rule

Funding Start follows a separate Window Loaded pattern. Its lookup continues to use `window.citiData.pageName`. Keep its event, conditions and values separate from the NGA outcome rules.

Funding Start setting	Existing implementation
Launch event	Core Window Loaded, order 50
Hostname	Existing condition for www.citicards.com and sit06, sit02 and sit07.accountonline.com
Path condition	Regular expression: ^(/cards/credit/application/flow\.action/?)\$
Page-name condition	%citidata_pageDef% matches ^(jJRSACQ_Secured_FundAcct)\$
Conversion	docid=dd_fundstart
Request values	window.prodId, window.prspectId, window.applId, window.businessTypCd, window.prodType, window.appType and window.sc
Readiness	Direct URL construction and script insertion; no App Start retry function

7. Part C: How the NGA rule triggers the pixels

The NGA Rule reads the application values and identifies the required business event. It then triggers the corresponding Direct Call. The matching BridgeTrack Launch rule creates the vendor request.

Application or analytics event

↓

NGA Rule

↓

Matching Direct Call

↓

BridgeTrack rule

↓

Vendor request

```

34  if (isValid) {
35      if (isRegistration) {
36          _satellite.track("NGA_application_registration");
37      } else if (isApproval) {
38          _satellite.track("NGA_application_approved");
39      } /*else if (isPendResolution) {
40          _satellite.track("NGA_application_pend_resolution");
41      }*/ else if (isDeclined) {
42          _satellite.track("NGA_application_declined");
43      } else if (isPended) {
44          _satellite.track("NGA_application_pended");
45      }
46  }
47  // GNADT - 8142 Start of the code

```

Figure 6. NGA Rule code calling the named application Direct Calls.

What you see in the screenshot

If the application is...	The NGA rule sends...
A registration	NGA_application_registration
Approved	NGA_application_approved
Declined	NGA_application_declined
Pended (pending)	NGA_application_pended

```
_satellite.track("NGA_application_approved");
```

The Approval Launch rule listens for NGA_application_approved and builds docid=NGA_Approved when its code checks qualify. The remaining events use their own identifiers and conversion values from Part B.

Signals the application must send

The NGA rule listens for two signals from the application. These replace the old Ensign names.

Old (Ensign)	New (Adobe Tags)	Meaning
na_tms_page_view	na_launch_page_view	A page view
na_tms_custom_event	na_launch_interaction	An interaction

The NGA Rule listens for na_launch_page_view and na_launch_interaction. It checks the application state and calls the specific business-event identifier. Dispatching an application Direct Call does not cause every BridgeTrack tag to fire.

IMPORTANT | Who changes this rule?

The NGA rule already exists. Edit it only when a new outcome has to be added, and ask your technical lead to review the change.

The application must send the event only after its data is ready. If the application sends it too early, the pixel may carry empty values.

8. Part D: Data Elements

A Data Element is a saved lookup. Instead of repeating the same lookup in many rules, you create it once and reuse it. This keeps rules short and easy to maintain.

Data Elements used by BridgeTrack

A Data Element is a reusable way to read a value needed by one or more rules. The source lookup names below are used by the existing conditions. The descriptive labels explain their purpose; they are not replacements for the existing names.

Source lookup	Purpose	Configuration
URL_cmp	Campaign query value	Core Query String Parameter; parameter name cmp
citidata_pageDef	Funding page name	Core Custom Code reading window.citiData.pageName, with its existing fallback

Create the campaign Data Element

1. Select Data Elements in the left menu. Open the existing lookup, or create it when the requirement needs a new lookup.
2. Use the existing lookup name URL_cmp when referencing the campaign value in the App Start condition.
3. Set Extension to **Core**.
4. Set Data Element Type to **Query String Parameter**.
5. In URL Query String Parameter Name, enter cmp. The existing lookup is case sensitive for the parameter name.
6. Select Save.

URL Query String Parameter Name (required)

☐ Allow capitalization differences (case-insensitive)

Figure 7. Campaign Data Element reading query-string parameter cmp.

TIP | Query key and campaign value

The parameter-name case setting applies to cmp. The separate App Start Value Comparison uses %URL_cmp% Starts With afa. Do not confuse the query-name setting with the comparison of its value.

A default-value setting does not validate a campaign prefix. Check that the lookup returns the expected cmp value and that the separate rule condition qualifies. Keep required storage and fallback settings with the agreed lookup.

Funding page-name lookup

citidata_pageDef reads window.citiData.pageName. When citiData exists but pageName is missing, it returns "na"; when citiData does not exist, it returns undefined. The Funding Start condition compares this lookup with jJRSACQ_Secured_FundAcct.

Using the Citi data layer

Citi is standardizing the use of window._dl for application and analytics values. Before updating a rule that reads window.citiData, identify the equivalent _dl field and verify its field name, expected value, data type where relevant and availability when the rule runs. The original NGA Ensign implementation already used _dl.pid and _dl.prospect_id. Launch retains those mappings; they are not a citiData-to-_dl change. Funding Start still reads window.citiData.pageName. Confirm the equivalent _dl mapping before changing that lookup.

- If a rule still uses `citiData`, find the matching `_dl` field first.
- Check that the field name, the value and its timing are correct before changing the reference.
- Ask your technical lead if you cannot find the matching field.

9. Part E: Test your pixel

Always test before publishing. Confirm the saved rule, Data Elements and dependencies are included in the required Development library and that its build has completed.

How to see the pixel in the browser

1. Open the UAT test page in Chrome or Edge, using the Development library. Your technical lead can share how your team loads it.
2. Press `F12` to open the developer tools.
3. Select the Network tab. Enable Preserve log if the journey navigates between pages.
4. In the filter box, type `bridgetrack`.
5. Refresh a qualifying page or follow the required application journey. For outcomes, check the exact matching Direct Call identifier.
6. Open the BridgeTrack request and inspect its URL and query parameters. Verify the endpoint, docid, pid, app, appId, type, r, ProspectID and sc. Check when the request occurred and compare the values with the application values at that event.
7. For App Start, also open the Console tab. The message "Pixel fired via recursive checker" should appear.

If a request is missing, check the event, conditions and required values. If more than one request appears, check rule execution and the application event count before changing the code. Also check navigation, SPA behaviour and applicable consent behaviour.

A browser request confirms the tag attempted the vendor request. Downstream conversion attribution is checked separately with the relevant media or account team.

What to check

Done	Check	What to verify
[]	Event	Correct page-load event or exact Direct Call identifier
[]	Conditions	Required hostname, path, query and internal business checks
[]	Data Elements	Correct source lookup and value returned at the event
[]	Application values	Correct _dl fields and global values for the specific rule
[]	Request	citi.bridgetrack.com/track/citicards/s/ and correct parameter casing
[]	Conversion values	Correct docid and all business values, including sc
[]	App Start	Six-field readiness; immediate check and retries when needed; appld behaviour retained
[]	Event count	Required tag fires at the correct business event; investigate unintended repeated requests
[]	Navigation and consent	SPA/full-page navigation and applicable consent behaviour

For App Start, check immediate execution when values exist, retries when a required value arrives later, and the warning when it remains missing. Also check the existing behaviour when appld is empty.

TIP

Save a screenshot of the passing check with the implementation record in Section 11.

10. Fixing common problems

What you see	Likely cause	What to do
Pixel does not fire	Wrong library, event or action	Confirm the Development build, rule event and required action.
Conditions do not qualify	Host, path, query or business value differs	Check the actual page values, regex setting and code checks.
Approved pixel does not fire	Direct Call identifier does not match	Compare the NGA Rule identifier with the Approval rule event.
Incorrect docid	Action copied from a different event	Compare the action value with the Part B mapping table.
Missing source values	Wrong field or value not available	Check the specific _dl/global field or Data Element at the event.

What you see	Likely cause	What to do
App Start reaches its retry limit	One of six required values remains missing	Check prodId, prspectId, sc, businessTypCd, prodType and appType.
Duplicate vendor requests	Repeated application events or multiple matching rules	Compare rule execution, Direct Calls and Network initiators.
Campaign condition fails	cmp key or afa comparison does not match	Check URL_cmp, query-name case and the separate Starts With condition.
Development changes not visible	Required revision or build is missing	Check the library resources, build result and page embed.
Old Ensign method is called	Executed code still uses Bootstrapper	Replace the required method with Launch or native JavaScript for that vendor.

If a problem is not listed here, send your technical lead the rule name, the page link, the time of the test and a screenshot of the Network tab.

11. Record your work

Fill in this table for every pixel you create. It helps the next person who has to support or change the pixel.

Field	Implementation details
Vendor	[Enter vendor/pixel]
Business requirement	[Enter event/conversion and page or journey]
Adobe Tags rule name	[Enter exact rule name]
Event type	[Enter selected Launch event]
Direct Call identifier	[Enter exact identifier, if applicable]
Conditions	[Enter interface conditions and internal code checks]
Source values	[Enter fields, globals/query values and their availability]
Data Elements	[Enter exact lookup names and configurations]
Vendor endpoint	[Enter host/path or library URL]
Vendor parameters	[Enter names, casing and value mappings]
Testing result	[Enter journey, expected/observed behaviour and result]
Implementation notes	[Enter relevant rule-specific details]

12. Adding a brand-new third-party pixel

Use this process for a new marketing or conversion requirement. BridgeTrack is one vendor example; select the new vendor's required method and values.

1 Understand the requirement

Identify the vendor, business event, page or journey, firing point, endpoint and required parameters.

2 Choose the Adobe Tags event

Choose Window Loaded or another suitable page-load event for page requirements, Direct Call for a named application/business event, or another appropriate Launch event.

3 Define the conditions

Use the hostname, path, query parameter, Data Element or application-state checks required by this requirement.

4 Find the source of every value

Map each vendor value to the correct `_dl` field, global, Data Element, query parameter or application value. Verify the business meaning before reusing a source.

5 Check when the values are ready

Use values directly when available. Add a suitable readiness check only when a required value arrives later. Do not add App Start retries to every tag.

6 Build the vendor request

Retain the vendor-defined endpoint, parameter names, casing, values and request structure. Use that vendor's parameters rather than BridgeTrack's.

7 Select the implementation method

Use an approved vendor extension where suitable, or the required Custom Code/native JavaScript. Native script creation applies when the vendor requires a script. Use Direct Calls where the application event requires them.

8 Test

Check the correct rule, event, conditions, values, request and firing time. Check unintended duplicate requests, navigation/SPA behaviour and applicable consent.

9 Document

Record the configuration and test result using the template in Section 11. Keep enough detail for another team member to maintain the tag.

NOTE

If one rule identifies several application states, keep that event rule separate from the matching vendor tag rules where required. Use the exact Direct Call identifiers between them.

Quick guide: which approach to use

Requirement	Use in Adobe Tags
Page-load tag	Appropriate page-load event, such as Window Loaded
Specific application or business event	Direct Call with the exact identifier
Reusable page, query or application value	Data Element
Vendor JavaScript file	Native script creation where required, or an approved vendor extension
One rule identifies several application states	Event rule plus matching Direct Call rules
Required value becomes available later	Readiness/retry check only where required

13. Golden rules

1. Keep the vendor request the same when the business need has not changed.
2. Use Adobe Tags or plain browser methods instead of Ensign functions.
3. Pick the event that matches the real business need.
4. Use the exact Direct Call identifier for application events.
5. Know the source of every value that the vendor needs.
6. Confirm the values are ready when the rule runs.
7. Keep the event rule and the vendor pixel rule separate when the design needs it.
8. Keep it simple, so the team can maintain it.
9. Reuse an existing pattern only when it fits the new requirement.

TIP | In short

Right event, right conditions, right values, right vendor request. Test it, then document it.

14. Who to contact

For	Contact
Rule or code questions	[Add technical lead name and email]
Access to Adobe Tags	[Add access owner]
Business requirement questions	[Add business owner]